Agentic Development

Working with AI coding agents as collaborators, not autocomplete

Davide Silva | Subvisual

Davide Silva

Lead Software Engineer · Subvisual

github.com/DavideSilva
blog.davidesilva.dev

The Problem

"I generated code with AI and it came out inconsistent."

The Problem

"I generated code with AI and it came out inconsistent."

- The agent doesn’t know your conventions

The Problem

"I generated code with AI and it came out inconsistent."

- The agent doesn’t know your conventions
- You paste the same setup prompt every session

The Problem

"I generated code with AI and it came out inconsistent."

- The agent doesn't know your conventions
- You paste the same setup prompt every session
- Without structure, the agent is a junior with no onboarding

The Core Idea

Don't just ask AI to generate code.

The Core Idea

Don't just ask AI to generate code.

The Core Idea

Don't just ask AI to generate code.

Structure your project so the agent is productive from the very first moment – just like you'd onboard a new developer.

The Building Blocks

Four essentials for a productive agent

The Building Blocks

Four essentials for a productive agent

01 Context files

The Building Blocks

Four essentials for a productive agent

01 Context files

02 Slash commands & skills

The Building Blocks

Four essentials for a productive agent

01 Context files

02 Slash commands & skills

03 Subagents

The Building Blocks

Four essentials for a productive agent

01 Context files

02 Slash commands & skills

03 Subagents

04 MCP servers

01 · Context Files

`CLAUDE.md`, `AGENTS.md` – the agent reads this on *every* session.

Put in

- Stack & architecture at a glance
- Coding conventions
- How to run, test, lint
- Non-obvious gotchas

Keep out

- Secrets & credentials
- Anything derivable from the code
- Ephemeral task notes
- Giant walls of text

01 · Context Files

`CLAUDE.md`, `AGENTS.md` – the agent reads this on *every* session.

Put in

- Stack & architecture at a glance
- Coding conventions
- How to run, test, lint
- Non-obvious gotchas

Keep out

- Secrets & credentials
- Anything derivable from the code
- Ephemeral task notes
- Giant walls of text

Think of it as the README you'd hand a new hire on day one.

01 · Context Files — Example

```markdown
# Project: Checkout Service

Stack: TypeScript · Fastify · Postgres · Bun

## Commands
- `bun dev` — run locally
- `bun test` — run tests (use this before claiming done)
- `bun run lint` — must pass before commit

## Conventions
- Prefer relative imports
- No hex/px values — use Tailwind tokens
- Errors bubble up; don't swallow with try/catch

## Gotchas
- Webhooks are idempotent — always check `event.id`
- Migrations run on deploy, not on boot
```

02 · Slash Commands & Skills

Reusable workflows the agent can invoke on demand.

Why

- Stop retyping the same instructions
- Standardize workflows across a team
- Turn tribal knowledge into muscle memory

Examples

- `/commit` – conventional commit message
- `/review-pr` – structured PR review
- `/new-migration` – scaffolded to your conventions

02 · Slash Commands & Skills

Reusable workflows the agent can invoke on demand.

Why

- Stop retyping the same instructions
- Standardize workflows across a team
- Turn tribal knowledge into muscle memory

Examples

- `/commit` – conventional commit message
- `/review-pr` – structured PR review
- `/new-migration` – scaffolded to your conventions

Slash commands – quick prompt shortcuts you type. **Skills** – richer units: metadata, scripts, files. The agent picks them up automatically when relevant.

02 · A Skill, concretely

```
---
name: new-migration
description: Use when the user asks for a new database migration.
---

# Creating a migration

1. Run `bun run db:new <name>` to scaffold the file
2. Write the `up` and `down` blocks — both are required
3. Never edit a migration that has already shipped
4. Update `schema.sql` after running it locally
5. Add a test that exercises the new columns
```

02 · A Skill, concretely

```
---
name: new-migration
description: Use when the user asks for a new database migration.
---

# Creating a migration

1. Run `bun run db:new <name>` to scaffold the file
2. Write the `up` and `down` blocks — both are required
3. Never edit a migration that has already shipped
4. Update `schema.sql` after running it locally
5. Add a test that exercises the new columns
```

The agent reads the `description`, decides when to invoke, and follows the steps — so every migration in the repo comes out the same way.

03 · Subagents

Specialized agents with their own context, tools, and system prompt.

Why

- Keep the main context clean
- Parallelize independent work
- Right tool for the job – not one generalist

Examples

- `code-reviewer` – reviews a diff against standards
- `explorer` – surveys unfamiliar codebases
- `test-runner` – runs & triages failures
- `security-auditor` – scans for OWASP issues

03 · Subagents

Specialized agents with their own context, tools, and system prompt.

Why

- Keep the main context clean
- Parallelize independent work
- Right tool for the job – not one generalist

Examples

- `code-reviewer` – reviews a diff against standards
- `explorer` – surveys unfamiliar codebases
- `test-runner` – runs & triages failures
- `security-auditor` – scans for OWASP issues

Reach for a subagent when the task is self-contained · would flood the main context · needs a focused prompt the main agent doesn't carry.

03 · A Subagent, concretely

```
---
name: code-reviewer
description: Review a diff against the project's coding standards.
  Use after a major feature is implemented, before committing.
tools: Read, Grep, Glob, Bash
---

You are a senior reviewer. Your job is to catch issues the author missed.

Focus on:
- Correctness &amp; edge cases
- Security (injection, auth, secrets)
- Consistency with conventions in CLAUDE.md
- Tests that actually exercise the change

Report findings as a prioritized list.
Do not fix anything — just review.
```

03 · A Subagent, concretely

```
---
name: code-reviewer
description: Review a diff against the project's coding standards.
  Use after a major feature is implemented, before committing.
tools: Read, Grep, Glob, Bash
---

You are a senior reviewer. Your job is to catch issues the author missed.

Focus on:
- Correctness &amp; edge cases
- Security (injection, auth, secrets)
- Consistency with conventions in CLAUDE.md
- Tests that actually exercise the change

Report findings as a prioritized list.
Do not fix anything — just review.
```

The main agent delegates; the reviewer runs in its own sandbox and returns a report. No review noise pollutes the main thread.

04 · MCP Servers

A protocol for giving the agent access to external tools & data.

Why

- The agent can *do* things, not just edit files
- Query your DB, hit your APIs, read designs
- Same interface across any MCP-aware client

Examples

- **Figma** – pull designs into code
- **Sanity** – query & mutate content
- **Playwright** – drive a real browser
- **Postgres / GitHub / Linear** – you name it

04 · MCP Servers

A protocol for giving the agent access to external tools & data.

Why

- The agent can *do* things, not just edit files
- Query your DB, hit your APIs, read designs
- Same interface across any MCP-aware client

Examples

- **Figma** – pull designs into code
- **Sanity** – query & mutate content
- **Playwright** – drive a real browser
- **Postgres / GitHub / Linear** – you name it

The tradeoff – each server eats context. More tools ≠ better agent. Install what the project actually needs.

04 · Wiring up an MCP server

```
# Remote HTTP server
claude mcp add --transport http figma https://mcp.figma.com/mcp

# Same pattern for any remote server
claude mcp add --transport http linear https://mcp.linear.app/mcp
```

Plan Before You Code

The fastest way to ship the wrong thing is to start typing.

Brainstorm → Spec → Plan → Code

Brainstorm → Spec → Plan → Code

Brainstorm

Explore intent & requirements
with the agent

Brainstorm → Spec → Plan → Code

Brainstorm

Explore intent & requirements *with* the agent

Spec

Write the design down. Commit it.

Brainstorm → Spec → Plan → Code

Brainstorm

Explore intent & requirements *with* the agent

Spec

Write the design down. Commit it.

Plan

Step-by-step implementation, reviewed before code

Brainstorm → Spec → Plan → Code

Brainstorm
Explore intent & requirements *with* the agent

Spec
Write the design down. Commit it.

Plan
Step-by-step implementation, reviewed before code

Code
Execute against the plan, tracked as todos

Brainstorm → Spec → Plan → Code

Brainstorm
Explore intent & requirements *with* the agent

Spec
Write the design down. Commit it.

Plan
Step-by-step implementation, reviewed before code

Code
Execute against the plan, tracked as todos

Brainstorm → Spec → Plan → Code

Brainstorm
Explore intent & requirements *with* the agent

Spec
Write the design down. Commit it.

Plan
Step-by-step implementation, reviewed before code

Code
Execute against the plan, tracked as todos

- **Brainstorming** surfaces assumptions before they become bugs
- **Plan mode** – the agent proposes, you approve, then it touches code
- Each step is cheap to redo. The code you'd have to rewrite isn't.

How much planning?

Match the ceremony to the size of the task.

How much planning?

Match the ceremony to the size of the task.

Small fix	Just do it. A one-line bug doesn't need a design doc.

How much planning?

Match the ceremony to the size of the task.

Small fix	Just do it. A one-line bug doesn't need a design doc.
New feature	Quick plan, skip the spec. Review the steps before executing.

How much planning?

Match the ceremony to the size of the task.

Small fix	Just do it. A one-line bug doesn't need a design doc.
New feature	Quick plan, skip the spec. Review the steps before executing.
Big refactor	Full flow: brainstorm → spec → plan → code. Commit each artifact.

How much planning?

Match the ceremony to the size of the task.

Small fix	Just do it. A one-line bug doesn't need a design doc.
New feature	Quick plan, skip the spec. Review the steps before executing.
Big refactor	Full flow: brainstorm → spec → plan → code. Commit each artifact.

The bigger the task, the more upfront planning pays off.

Putting It Together

One feature, four building blocks.

Feature from a Figma design

Designer drops a Figma link. "Build this."

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"

→ reads `CLAUDE.md`
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
→ reads `CLAUDE.md`
→ `get_design` via Figma MCP
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
→ reads `CLAUDE.md`
→ `get_design` via Figma MCP
→ `/design-to-component`
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
→ reads `CLAUDE.md`
→ `get_design` via Figma MCP
→ `/design-to-component`
→ dispatches `code-reviewer`
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
→ reads `CLAUDE.md`
→ `get_design` via Figma MCP
→ `/design-to-component`
→ dispatches `code-reviewer`
→ `/commit`
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
→ reads `CLAUDE.md`
→ `get_design` via Figma MCP
→ `/design-to-component`
→ dispatches `code-reviewer`
→ `/commit`
✓ PR ready
```

Feature from a Figma design

Designer drops a Figma link. "Build this."

```
$ "build this Figma design"
→ reads `CLAUDE.md`
→ `get_design` via Figma MCP
→ `/design-to-component`
→ dispatches `code-reviewer`
→ `/commit`
✓ PR ready
```

No copy-paste. No re-explaining. The project tells the agent how to work here.

Pitfalls

Easy ways to sabotage yourself.

What not to do

What not to do

The 500-line CLAUDE.md

Nobody maintains it. It rots. Keep it tight – onboarding, not a manual.

What not to do

The 500-line CLAUDE.md

Nobody maintains it. It rots. Keep it tight – onboarding, not a manual.

MCP buffet

Every tool eats context. More servers ≠ smarter agent. Install what the project needs.

What not to do

The 500-line CLAUDE.md

Nobody maintains it. It rots. Keep it tight – onboarding, not a manual.

MCP buffet

Every tool eats context. More servers ≠ smarter agent. Install what the project needs.

Autocomplete mindset

Skipping structure, then blaming the agent for inconsistency. Onboarding first.

What not to do

The 500-line CLAUDE.md

Nobody maintains it. It rots. Keep it tight – onboarding, not a manual.

MCP buffet

Every tool eats context. More servers ≠ smarter agent. Install what the project needs.

Autocomplete mindset

Skipping structure, then blaming the agent for inconsistency. Onboarding first.

Uncommitted context

Your teammate's agent doesn't know what yours knows. Commit the context files.

Takeaway

Takeaway

Takeaway

Treat your project as the agent's onboarding doc.

Takeaway

Treat your project as the agent's onboarding doc.

Start small: one `CLAUDE.md`, one skill, one subagent.

Takeaway

Treat your project as the agent's onboarding doc.

Start small: one `CLAUDE.md`, one skill, one subagent.

The more structure you give it, the more leverage you get back.

Tools Worth Knowing

Some I use. Some are on my radar.

A few worth a look

A few worth a look

Superpowers

Claude Code plugin

Ships ready-made skills: brainstorming, planning, TDD, debugging, code review. Drop-in process for working with the agent.

github.com/obra/superpowers →

A few worth a look

Superpowers

Claude Code plugin

Ships ready-made skills: brainstorming, planning, TDD, debugging, code review. Drop-in process for working with the agent.

github.com/obra/superpowers →

Agent Deck

TUI · multi-agent control

Mission control for many sessions at once: status, session forking, MCP & skill management, git worktrees, Docker sandboxes.

github.com/asheshgoplani/agent-deck →

A few worth a look

Superpowers

Claude Code plugin

Ships ready-made skills: brainstorming, planning, TDD, debugging, code review. Drop-in process for working with the agent.

github.com/obra/superpowers →

Agent Deck

TUI · multi-agent control

Mission control for many sessions at once: status, session forking, MCP & skill management, git worktrees, Docker sandboxes.

github.com/asheshgoplani/agent-deck →

Headroom

Context compression

Sits between agent and LLM, compresses tool outputs & file reads. Fewer tokens, same answers – runs locally as a proxy or SDK.

github.com/chopratejas/headroom →

A few worth a look

Superpowers

Claude Code plugin

Ships ready-made skills: brainstorming, planning, TDD, debugging, code review. Drop-in process for working with the agent.

github.com/obra/superpowers →

Agent Deck

TUI · multi-agent control

Mission control for many sessions at once: status, session forking, MCP & skill management, git worktrees, Docker sandboxes.

github.com/asheshgoplani/agent-deck →

Headroom

Context compression

Sits between agent and LLM, compresses tool outputs & file reads. Fewer tokens, same answers – runs locally as a proxy or SDK.

github.com/chopratejas/headroom →

Sandboxed environments

Practice, not a single tool

Run agents in containers or with restricted permissions – Claude Code's sandbox mode, devcontainers, Docker. Let it move fast without the blast radius.

A few worth a look

Superpowers

Claude Code plugin

Ships ready-made skills: brainstorming, planning, TDD, debugging, code review. Drop-in process for working with the agent.

github.com/obra/superpowers →

Agent Deck

TUI · multi-agent control

Mission control for many sessions at once: status, session forking, MCP & skill management, git worktrees, Docker sandboxes.

github.com/asheshgoplani/agent-deck →

Headroom

Context compression

Sits between agent and LLM, compresses tool outputs & file reads. Fewer tokens, same answers – runs locally as a proxy or SDK.

github.com/chopratejas/headroom →

Sandboxed environments

Practice, not a single tool

Run agents in containers or with restricted permissions – Claude Code's sandbox mode, devcontainers, Docker. Let it move fast without the blast radius.

Thanks

Questions?

github.com/DavideSilva
blog.davidesilva.dev